

— Web apps

Chapter 2: JSX code

L. Espeel



— Inhoud

- In dit hoofdstuk:
 - Using the `<React.Fragment>`, `<Fragment>` and the `<>` Tag
 - Inserting JavaScript Code into JSX
 - Writing Conditions in JSX
 - Writing a Loops in JSX
 - Using Styles in JSX

— <React.Fragment>, <Fragment> en <> tag

Het startpunt in de codevoorbeelden is *App.js* die *App()*-functie bevat waarin JSX-code wordt geretourneerd. Het is steeds verplicht om na een return statement een **root-element** op te geven, dit kan een `<div>` zijn.

- Maar is meer overhead: er wordt een extra div-element in DOM geplaatst.
- Beter is één van de 3 volgende React-tags te gebruiken.

Deze tags zijn synoniem van elkaar en geven dezelfde uitvoer, doch kleine verschillen:

- `<React.Fragment>` tag vereist `import React from "react";`
- `<Fragment>` tag vereist `import { Fragment } from "react";`
- `<>` tag vereist geen import en is de kortste vorm, kan wel niet gebruikt worden indien je attributen wilt aanmaken.

3

— Conditie in JSX

Conditie in JSX kan je gebruiken d.m.v.:

- Zelfoproepende JavaScript-functie:

```
{
  (function () {
    if (end !== 0 && count >= end) return <b>Counter stopped</b>;
    else return <i>Counter in progress</i>;
  })()
}
```

- JavaScript ternaire operator:

```
{
  (end !== 0 && count >= end) ? <b>Counter stopped</b> : <i>Counter in progress</i>
}
```

4

— Lussen in JSX ⁽¹⁾

Lussen in JSX kan je gebruiken d.m.v.:

- Zelfoproepende JavaScript-functie.

- Opmerking: er is hier nog verfijning nodig (zie warning in console).

(Voeg een JSX-element *React.Fragment* of *Fragment* toe met attribuut **key** dat een unieke waarde moet bevatten, bv. de waarde *i*).

```
{
  (function () {
    const jsx = [];

    for (let i = 0; i < 3; i++) {
      jsx.push(
        <>
          Counter {i} defined by {`<Counter init='5' end='${10 + i}' />`}
          <br />
          <Counter init='5' end={10 + i} />
          <br /><br />
        </>
      );
    }

    return jsx;
  })()
}
```

5

— Lussen in JSX ⁽²⁾

- *map()*-functie op een JavaScript-array.

- Bij (grote) arrays met sequenties is dit efficiënter:

- [...Array(10).keys()]
→ waarden 0 t.e.m. 9
- [...Array(10).keys()].slice(1)
→ waarden 1 t.e.m. 10

```
{
  [0, 1, 2].map(function (i) {
    return (
      <React.Fragment key={i}>
        Counter {i} defined by {`<Counter init='5' end='${10 + i}' />`}
        <br />
        <Counter init='5' end={10 + i} />
        <br /><br />
      </React.Fragment>
    )
  })
}
```

6

— Stijlen in JSX (1)

In JSX-code kunnen er ook CSS-stijlen toegevoegd worden:

```
return (  
  <>  
    Initial value of the counter is: {init}  
    <br />  
    End of the counter at: {end}  
    <br />  
    The counter is: {count}  
    <br />  
    {  
      (end !== 0 && count >= end) ? <b style={{ color: "red", fontSize: "20px" }}>Counter stopped</b>  
      : <i>Counter in progress</i>  
    }  
  </>  
)
```

7

— Stijlen in JSX (2)

Wat als we nu die voorgaande inline CSS styling willen vervangen door externe CSS styling (in een apart bestand)?

Algemene stappen:

- CSS-bestand aanmaken met hierin de kleur en lettergrootte onder een CSS-klasse naam.
- Het CSS-bestand vervolgens importeren in *Counter.js*.
- Attribuut *className* gebruiken in het element.

Exacte syntax: zie les.

8